Kadanes Algorithm

AlgoExpert · Medium · Famous Algorithms

The problem

- Maximum sum of a non-empty contiguous subarray.

Intuition

- The best subarray ending here is either this element alone or it extends the best ending at the previous index.

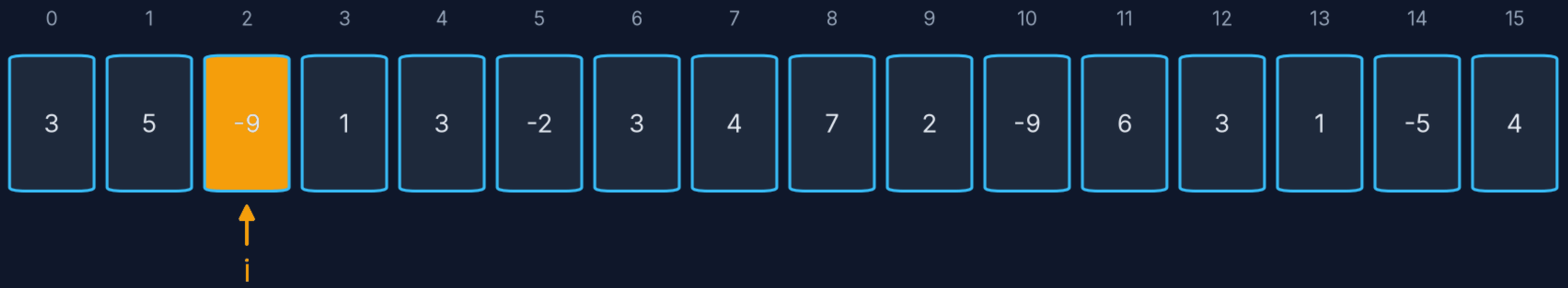
Brute force vs optimal

- Brute force: all subarrays → $O(n^2)$ ($O(n^3)$ naive)
- Divide and conquer → $O(n \log n)$
- Kadane: best ending here → $O(n)$, $O(1)$

Approach

- 1. Running best ending here
- 2. Max of element or element plus running
- 3. Track the global max

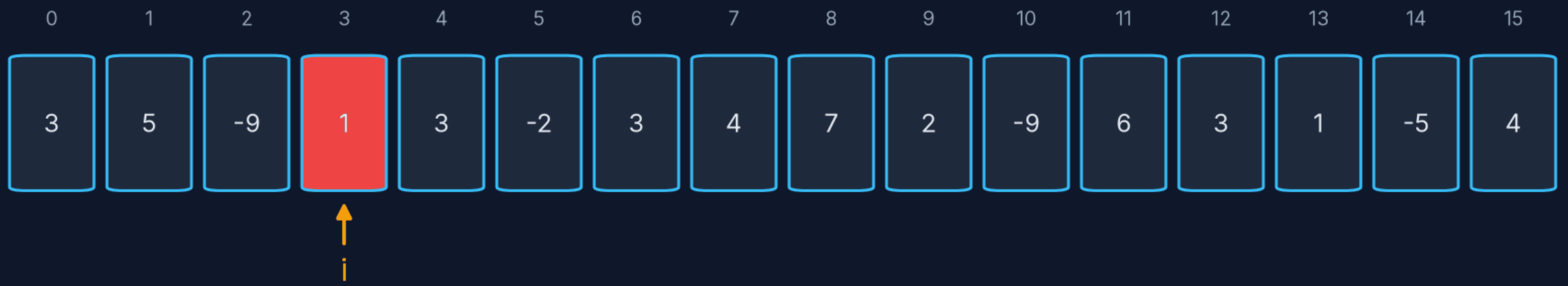
Worked example




```
best ending here = -1
best overall = 8
```

i=2: cur = -1, best = 8

Worked example




```
best ending here = 1 (restart)
best overall = 8
```

i=3: cur = 1, best = 8

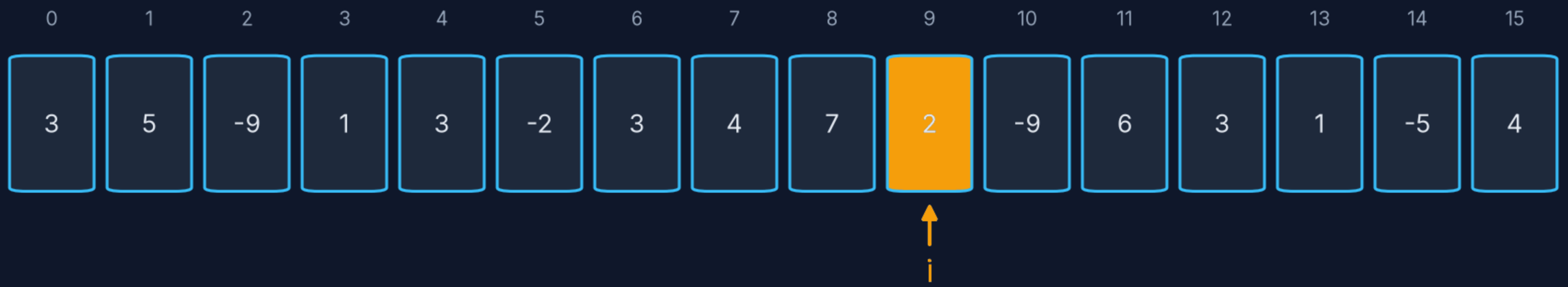
Worked example




```
best ending here = 16
best overall = 16
```

i=8: cur = 16, best = 16

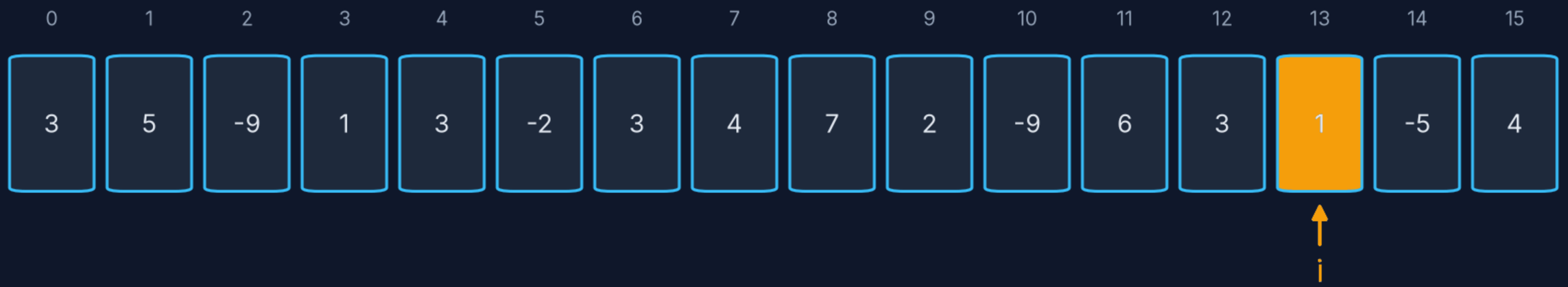
Worked example




```
best ending here = 18
best overall = 18
```

i=9: cur = 18, best = 18

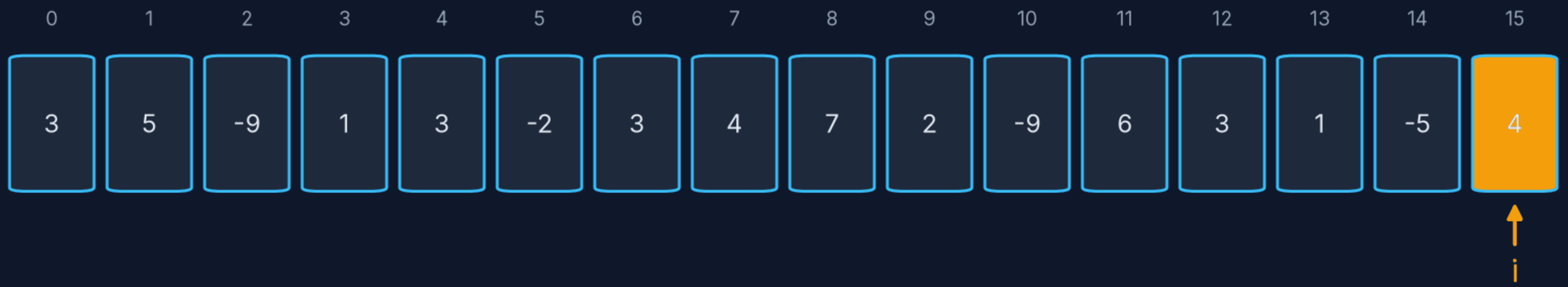
Worked example




```
best ending here = 19
best overall = 19
```

i=13: cur = 19, best = 19

Worked example




```
best ending here = 18
best overall = 19
```

i=15: cur = 18, best = 19

Java solution

```java
public class KadanesAlgorithm {
    public static int kadanesAlgorithm(int[] a) {
        int cur = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) { cur = Math.max(a[i], cur + a[i]); best = Math.max(best, cur); }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- All negative → the largest single element
- Initialise with a[0], not 0
- Overflow with large values

Interview tips & follow-ups

- Follow-up: return the indices
- Follow-up: circular array, 2D max submatrix

Recap

- The best subarray ending here is either this element alone or it extends the best ending at the previous index.
- Complexity: O(n)